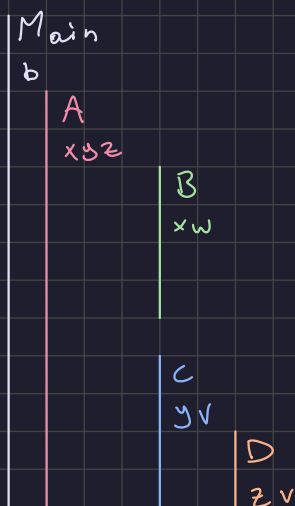


1

```

{
  int b;
  void A() {
    int x = 0, y = 0, z = 0;
    void B() {
      int x = 3, w = 3;
      x = y + z;
    }
    void C() {
      int y = 1, v = 1;
      void D() {
        int z = 2, v = 2;
        B();
        v = x + y;
      }
      D();
      z = z + y;
    }
    C(7);
  }
  A();
}

```



$SD(main) = 0$
 $SD(A) = 1$
 $SD(B) = SD(C) = 2$
 $SD(D) = 3$

main $\rightarrow$ A $\rightarrow$ C $\rightarrow$ D $\rightarrow$ B

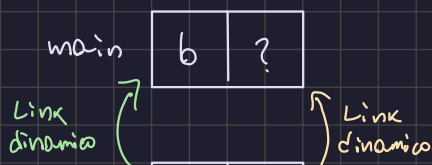
Scoping statico : Esecuzione ed evoluzione dello stack

I concetti che vanno definiti sono:

- Cosa distingue scoping statico e dinamico + definizione
- Quando servono le regole di scoping
- Definizione di profondità statica (SD) e spiegazione per ogni procedura
- Definizione di link statico (catena statica) e dinamico (catena dinamica)
- Definizione di CRT (cos'è e come si gestisce)

Link statico

chiamata main $\rightarrow$ A



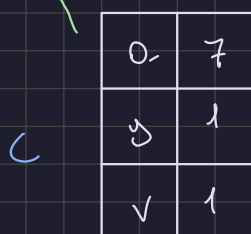
calcolo del link statico:

$$\begin{aligned}
 k &= SD(\text{Chiamante}) - SD(\text{Chiamato}) + 1 \\
 \Rightarrow k_A &= SD(\text{main}) - SD(A) + 1 \\
 &= 0 - 1 + 1 = 0
 \end{aligned}$$

$$\Rightarrow CS(A) = \text{Indirizzo}(\text{main}) = \text{main}$$

k rappresenta di quanto bisogna risalire la catena statica per trovare l'RdA del genitore statico

chiamata A $\rightarrow$ C(7)



calcolo del link statico:

$$\begin{aligned}
 k &= SD(\text{Chiamante}) - SD(\text{Chiamato}) + 1 \\
 \Rightarrow k_A &= SD(A) - SD(C) + 1 \\
 &= 1 - 2 + 1 = 0
 \end{aligned}$$

$$\Rightarrow CS(C) = A$$

chiamata C → D

D

z	2
v	2

chiamata D → B

B

w	3
x	3

calcolo del link statico:

$$\begin{aligned}k &= SD(\text{Chiamante}) - SD(\text{Chiamato}) + 1 \\ \Rightarrow k_A &= SD(C) - SD(D) + 1 \\ &= 2 - 3 + 1 = 0\end{aligned}$$

$$\Rightarrow CS(C) = C$$

calcolo del link statico:

$$\begin{aligned}k &= SD(\text{Chiamante}) - SD(\text{Chiamato}) + 1 \\ \Rightarrow k_A &= SD(D) - SD(B) + 1 \\ &= 3 - 2 + 1 = 2\end{aligned}$$

$$\Rightarrow CS(C) = 2$$

esecuzione di B: $x = y + z$

La x da aggiornare si trova in B

y e z non sono locali, quindi bisogna trovarle nello stack e di conseguenza bisogna calcolare N:

$$N_a = SD(\text{Procedura che usa } a) - SD(\text{Procedura più vicina che definisce } a \text{ nello schema di annidamento})$$

N_x dice di quanto si deve risalire la catena statica per risolvere x non locale

$$N_y = SD(B) - SD(A) = 2 - 1 = 1$$

$$N_z = SD(B) - SD(A) = 1$$

Quindi si ha che

$$x = 5 + 0$$

L'RdA di B diventa:

B

x	5
w	3

E poi viene rimosso dallo stack. Lo stato diventa quindi:



Esecuzione di D: $v = x + y$

v è locale in D, mentre

- x è in A e vale 0
- y è in C e vale 1

$$N_x = SD(D) - SD(A) = 3 - 1 = 2$$

$$N_y = SD(D) - SD(C) = 1$$

Quindi si ha

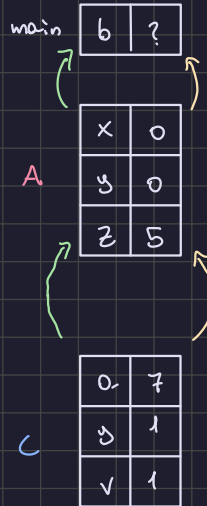
$$v = 0 + 1 = 1$$

L'RdA di D diventa:

D

z	2
v	1

E viene rimosso dallo stack



Esecuzione di C $x = z + y$

y è locale in c

$$N_x = SD(C) - SD(A) = 2 - 1 = 1$$

$$N_z = SD(C) - SD(A) = 1 \Rightarrow z = 5$$

Quindi si ha:

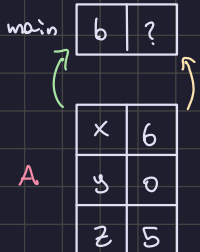
$$x = 5 + 1$$

L'RdA di A diventa

A

x	6
y	0
z	5

E l'Rda di C viene rimosso dallo stack



Successivamente avviene la terminazione di tutte le procedure e la distruzione degli RdA

Scoping dinamico: Esecuzione e CRT

1. main → A

b	→	main, ?
x	→	A, 0
y	→	A, 0
z	→	A, 5
v		
w		

2. A → C

b	→	main, ?
x	→	A, 0 6
y	→	C, 1 → A, 0
z	→	A, 5
v	→	C, 1
w		

3. C → D

b	→	main, ?
x	→	A, 0
y	→	C, 1 → A, 0
z	→	D, 2 → A, 5
v	→	D, 2 1 → C, 1
w		

4. D → B

b	→	main, ?
x	→	B, 3 3 → A, 0
y	→	C, 1 → A, 0
z	→	D, 2 → A, 5
v	→	D, 2 → C, 1
w	→	B, 3

$$x = z + y = 6$$

5 1

← uscita da D
esecuzione di C

$$v = x + y = 1$$

0 1

← uscita da B
esecuzione di D

$$x = y + z = 3$$

1 2

7. Esecuzione di C

6. Esecuzione di D

5. Esecuzione di B